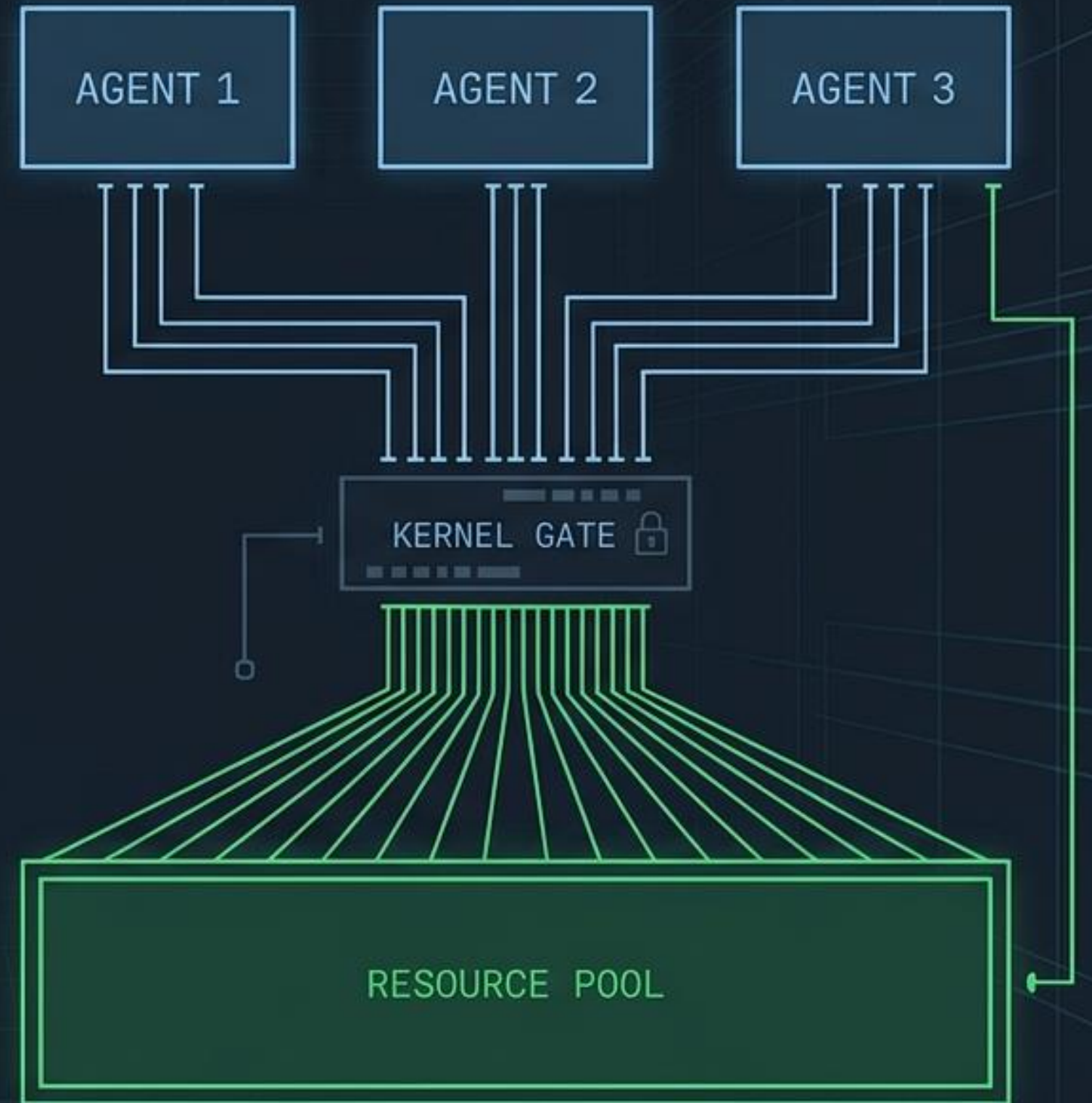


Sentari Agent-OS

An Operating-System-Inspired
Kernel for Scheduling, Isolation,
and Deadlock Detection in
Multi-Agent AI Systems

Author: Vishal Sharma
Roll No: 25225039
Evaluation: C1 Specialization Project



The Chaos of Current AI

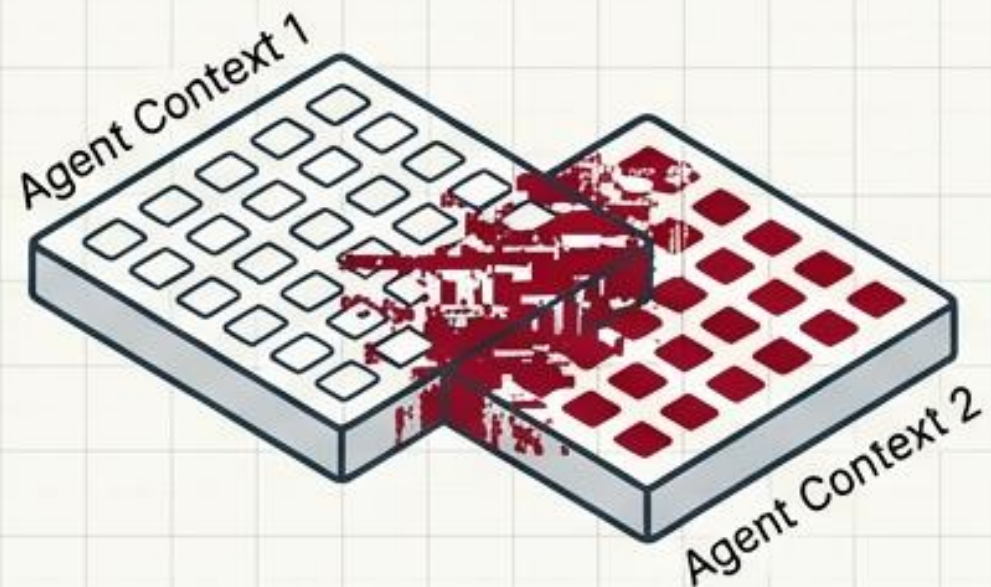
No Resource Governance

Unbounded Consumption. Any agent can consume unlimited API calls or tokens, starving others and destroying budgets.



No Isolation

Shared Mutability. A hallucinated or corrupted value from one agent silently propagates into another's context.



No Deadlock Handling

Unhandled Hangs. If Agent A waits on B, and B waits on A, the entire system hangs without cycle detection.



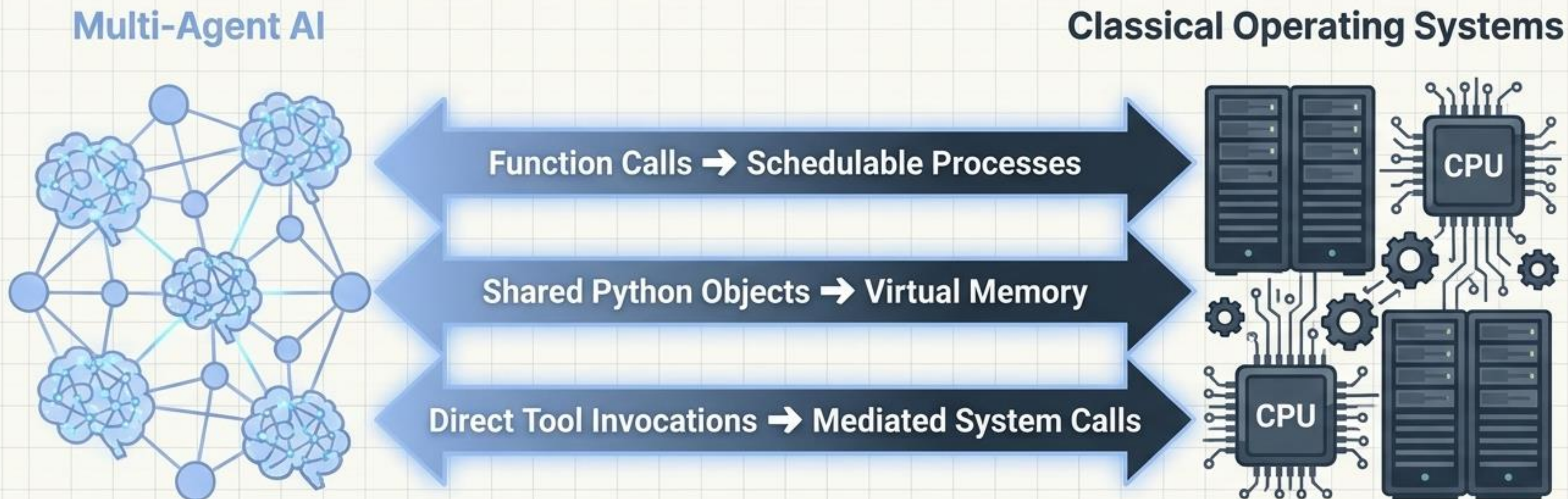
No Mediated Access

Direct Execution. Agents call external tools and APIs directly without centralized validation, sandboxing, or audit trails.



The Paradigm Shift: Agents Require a Kernel

Today's multi-agent frameworks wire agents together purely at the application layer. Agent-OS introduces a strict middleware kernel that models every AI agent as a scheduled, resource-bounded, and isolated process.

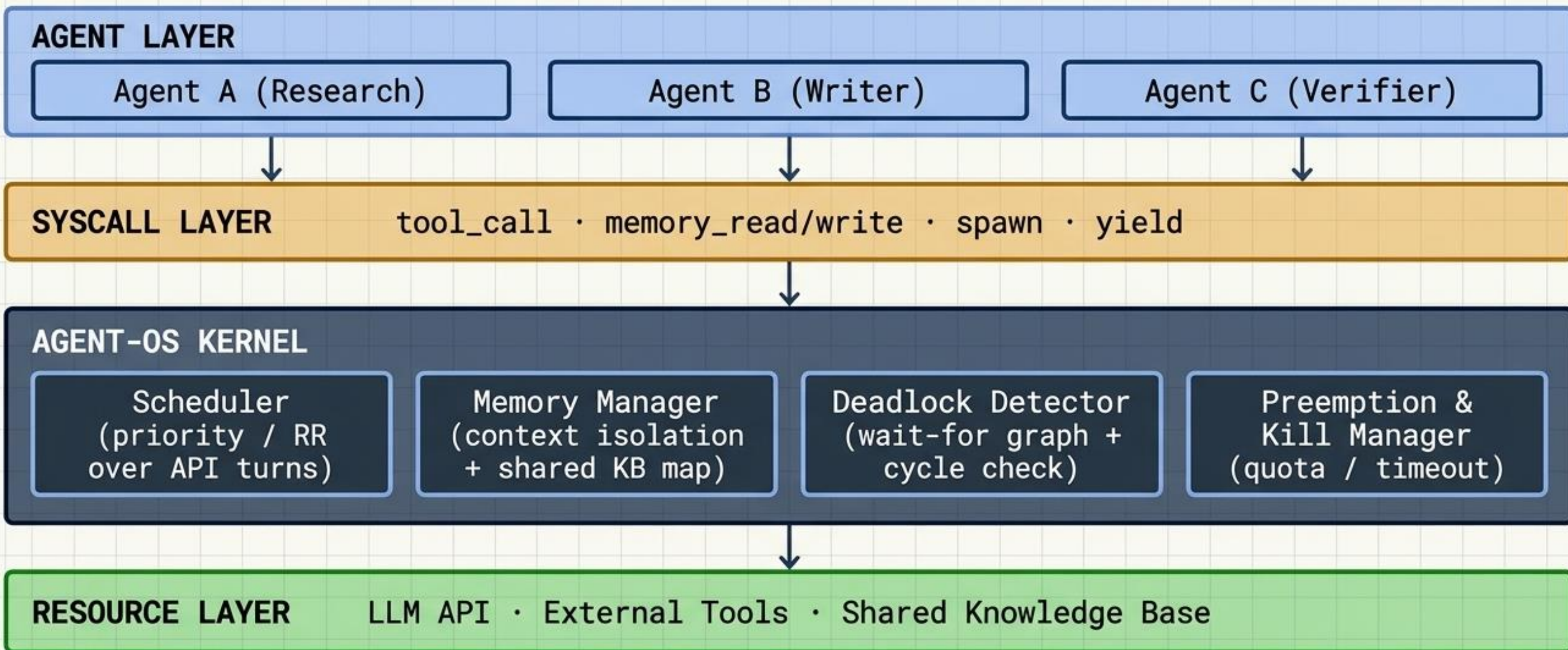


The Translation Matrix: From Chaos to Orchestration

	Current Frameworks (The Problem)	Agent-OS (The Solution)	Classical OS Concept (The Root)
Resource Governance	Unbounded API usage	Preemptive Quotas ✓	CPU Scheduling
Context Integrity	Shared Mutability ✗	Memory Isolation ✓	Virtual Memory
Inter-Agent Dependency	Unhandled Hangs ✗	Cycle Detection ✓	Wait-for Graphs
Tool Execution	Direct Execution ✗	Syscall Mediation ✓	Kernel Space
Lifecycle Control	Infinite Loops ✗	Forced Kills ✓	Process Signals

Agent-OS Architecture

Every interaction is a mediated, logged call. No layer is permitted to skip the one below it.



The Anatomy of an Agent: The Process Control Block (PCB)

Without a PCB, there is no entity to govern. The PCB is the foundational data structure of Agent-OS.

Agent ID: a8f7-9c12

state: **READY**

priority: 1

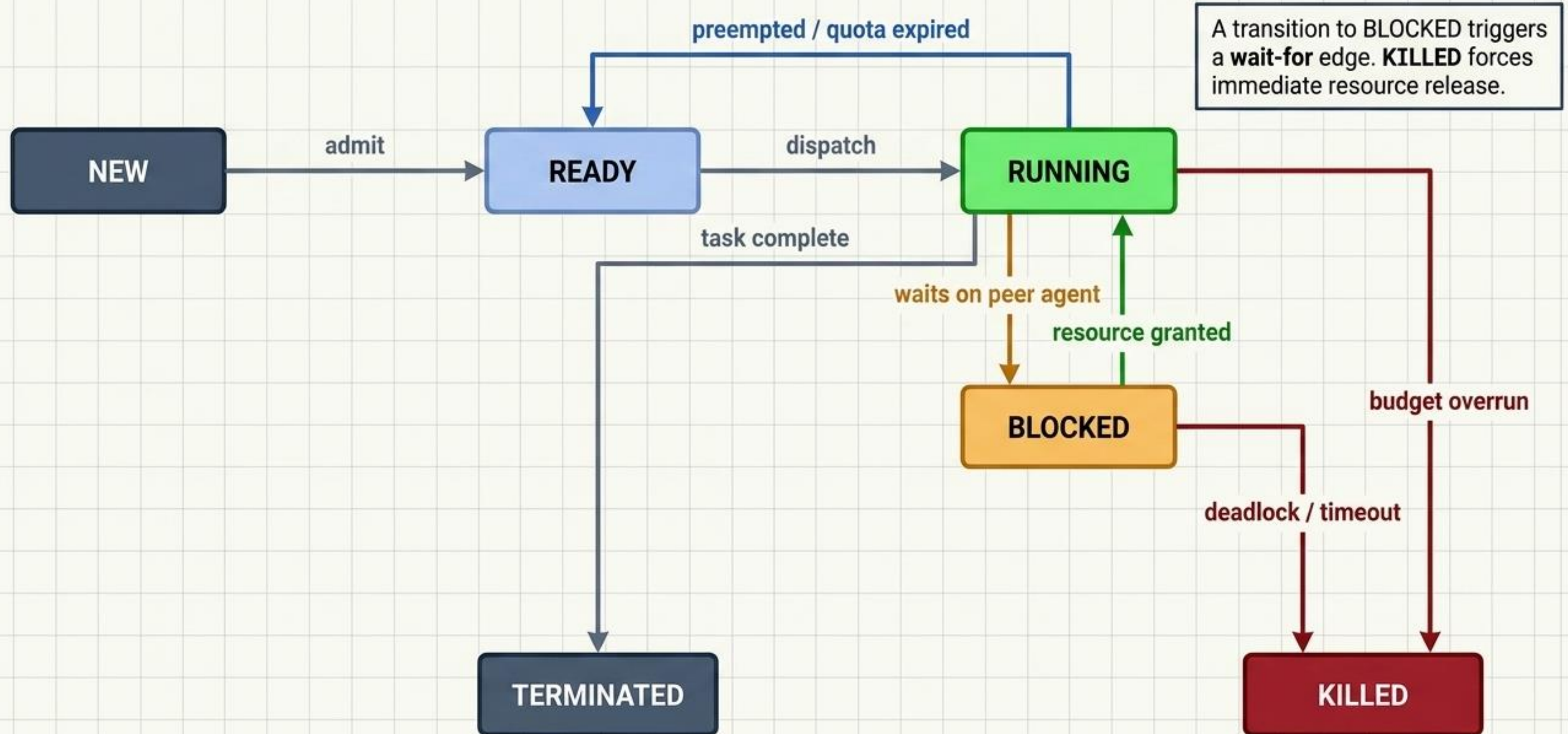
quota_total: **5000**

quota_used: **1250**

parent_id: null

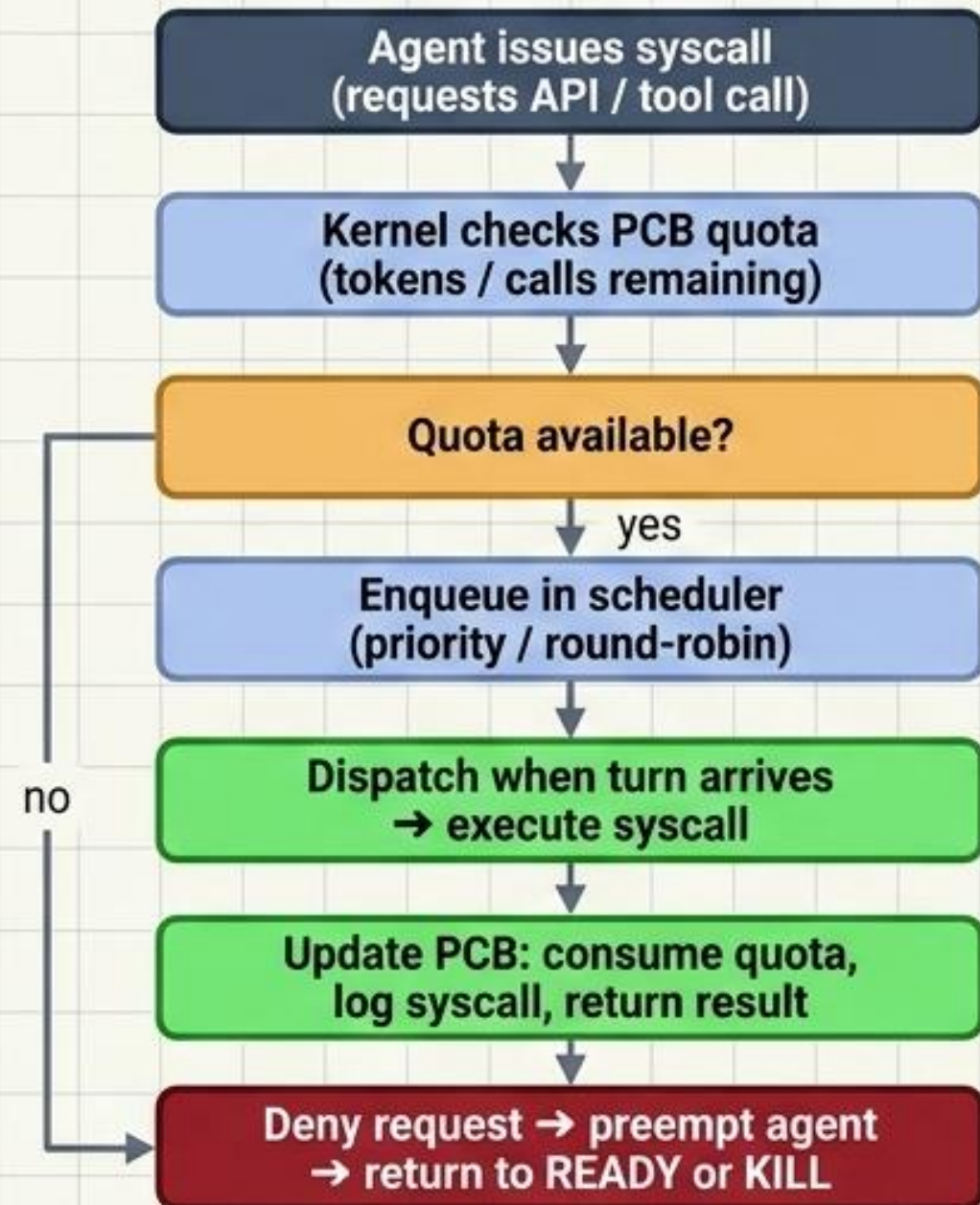
- **State Tracking:** Transforms a stateless LLM prompt into a lifecycle-managed entity.
- **Priority Scheduling:** Enables kernel to age processes and prevent starvation.
- **Hard Budgets:** Tracks token and API call consumption down to the exact unit.

Agent Process Control Block (PCB) – Lifecycle States



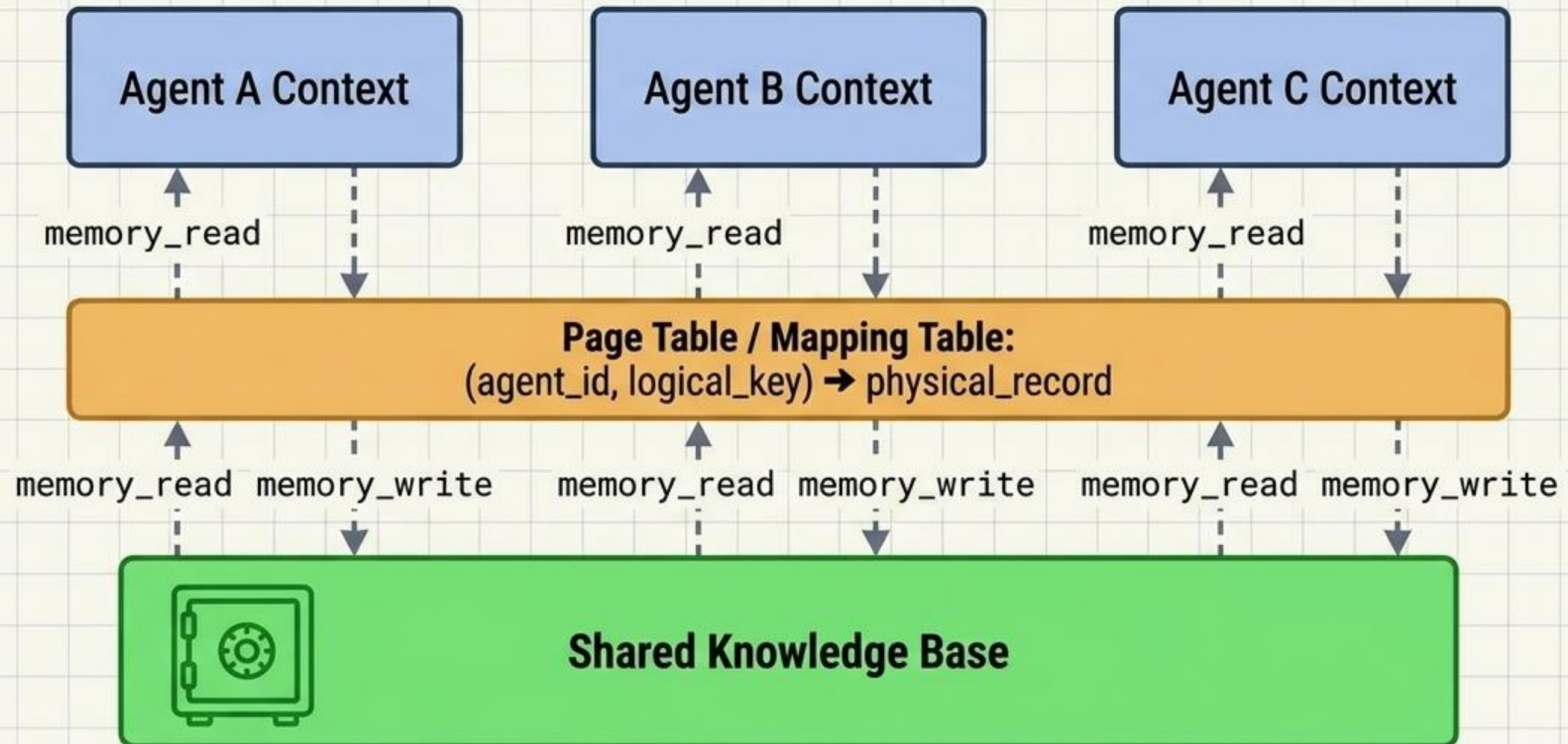
Resource-Aware Scheduling Pipeline

Time-slicing applied to LLM tokens. By enforcing strict quotas before dispatch, the scheduler guarantees fair API access and prevents single-agent resource monopolization.



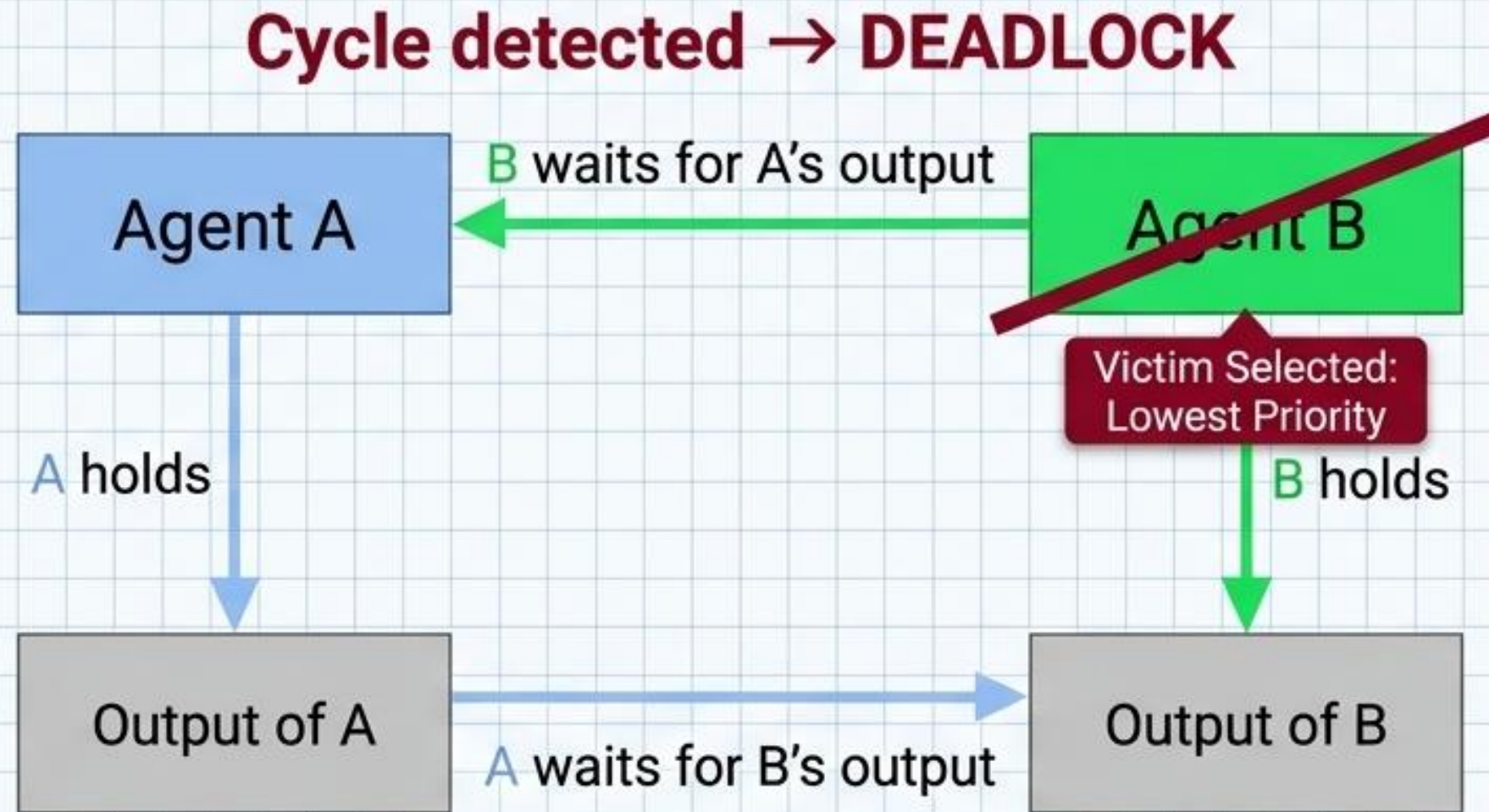
Memory Manager: Context Isolation & Shared Knowledge

Prevents one agent's hallucinated state from silently corrupting another. Access is explicit, mapped, and controlled.



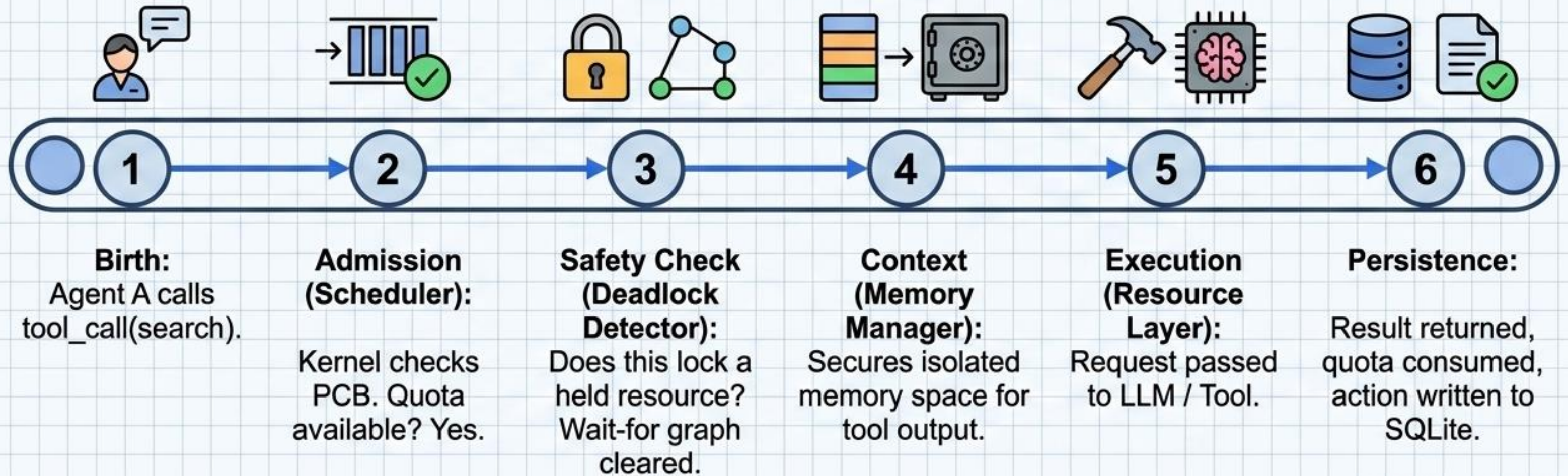
Deadlock Detection Between Agents

The module runs an $O(V + E)$ **depth-first search** on **every new wait-for edge**. When a cycle is caught, the **Preemption Manager** terminates the **lowest priority agent** to break the deadlock.



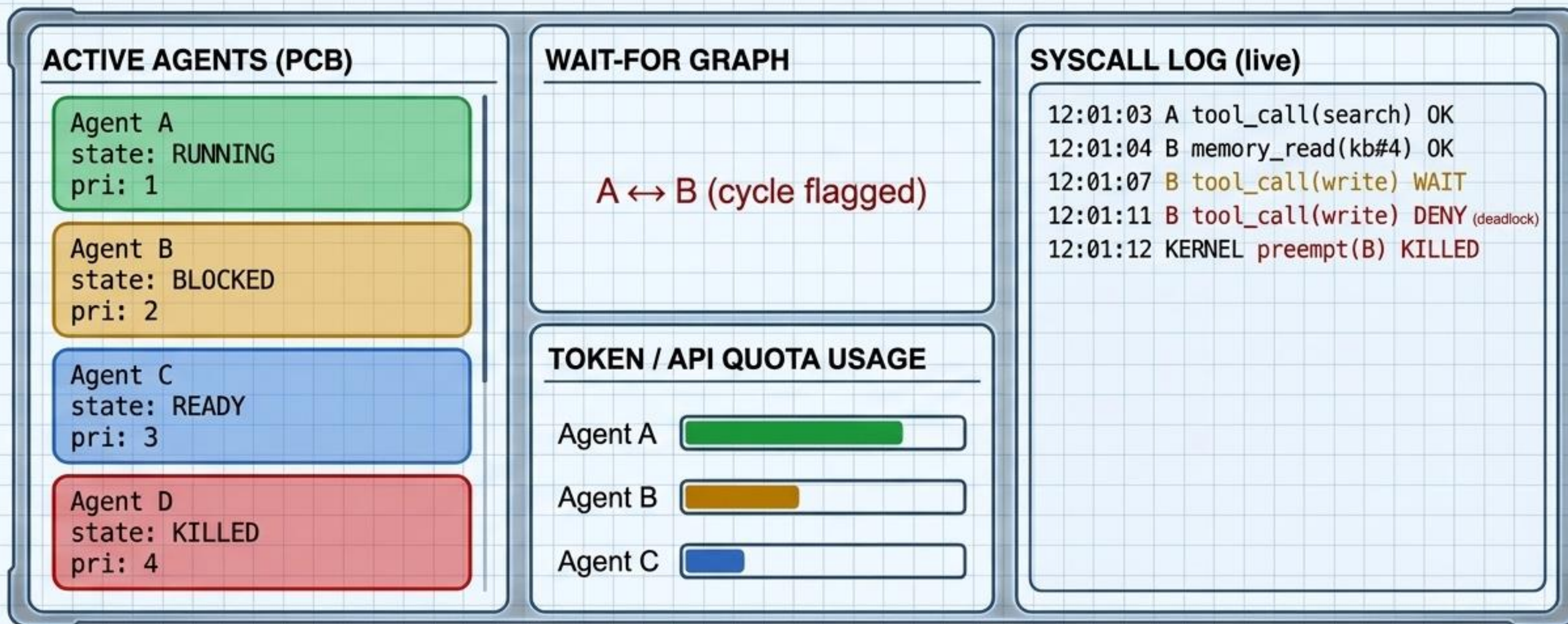
Synthesis: The Lifecycle of a Syscall

Every disparate module cooperates in milliseconds to validate a single action, proving that agent orchestration is fundamentally an operating system problem.



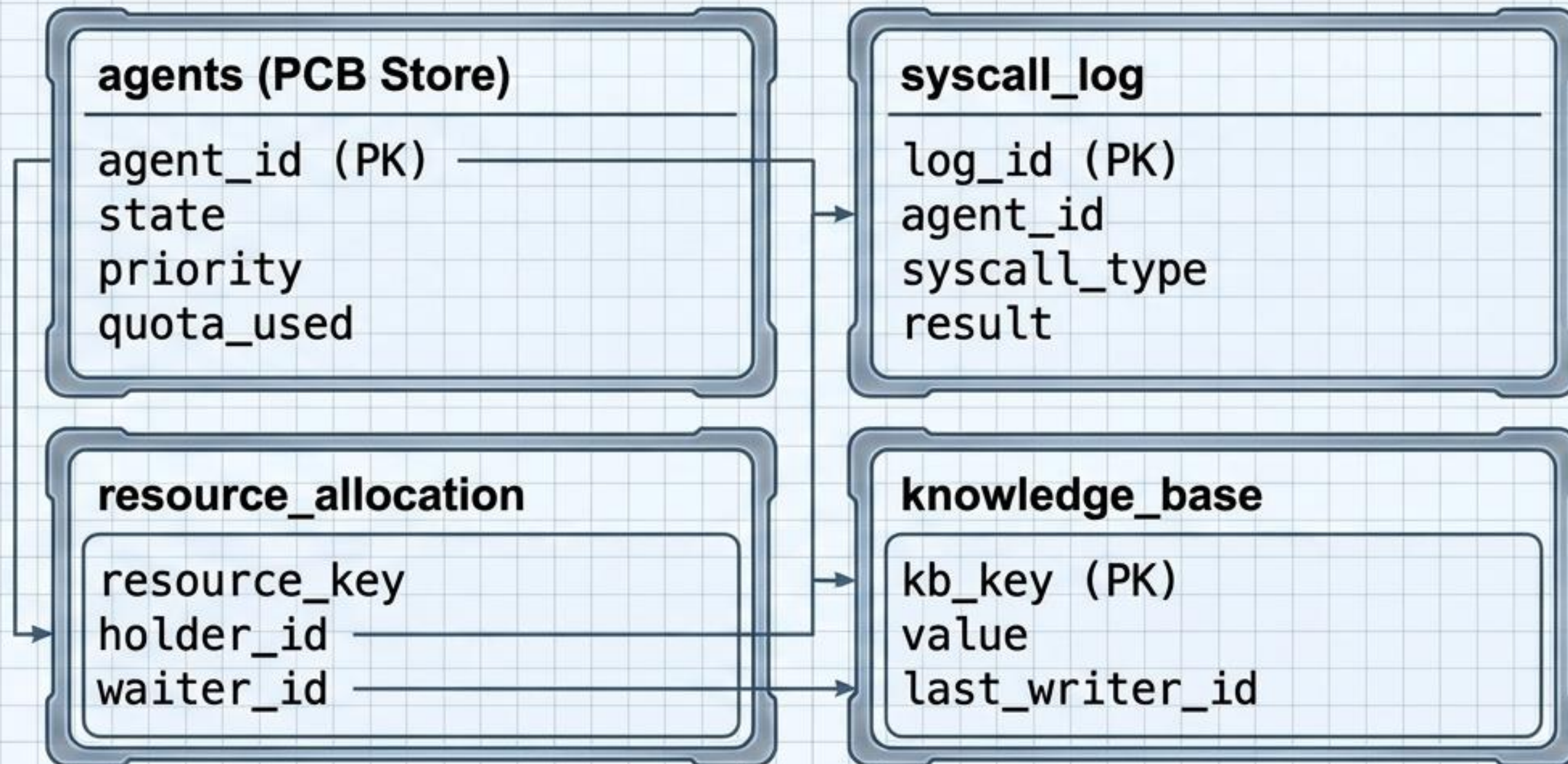
AGENT-OS — CONTROL DASHBOARD

The dashboard is a read-only observability surface. It reveals the invisible orchestration happening within the kernel space.



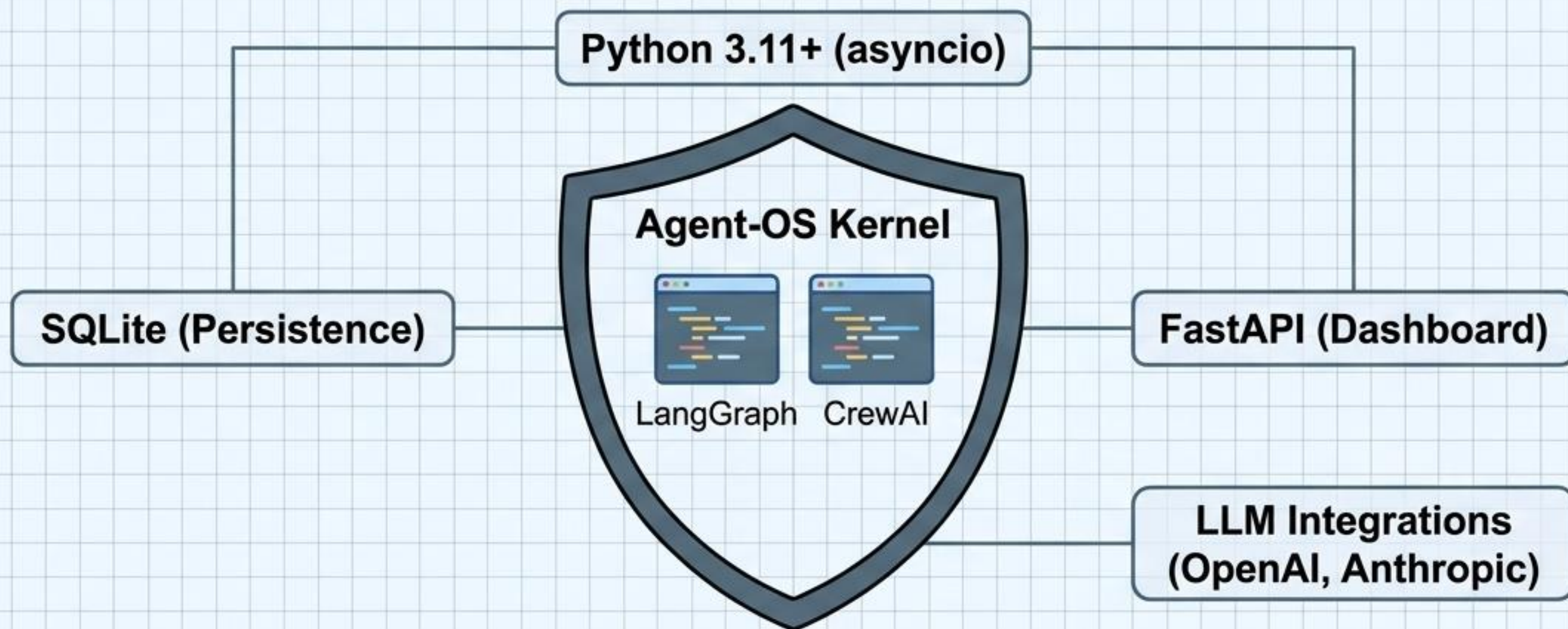
Perfect Auditability: The Persistence Layer (SQLite)

Because every action is a mediated syscall, Agent-OS provides a mathematically complete audit trail of the AI system's entire history.



Tech Stack & Seamless Middleware Integration

Agent-OS does not replace your orchestration logic. It acts as a middleware intercept layer. By routing your framework's tool-execution steps through our Python syscall interface, you gain kernel-level governance with zero framework rewrite.



The Expected Impact: Resilient & Responsible AI

SDG 9: Industry, Innovation & Infrastructure



Building foundational software infrastructure to make multi-agent deployments resilient, auditable, and safe for industrial adoption.

SDG 12: Responsible Consumption & Production



By enforcing token quotas and preempting runaway agents, Agent-OS directly stops unbounded API execution, reducing wasted compute and energy footprint.